Day 7 Evidence Workbooklet Instructor Key

Ordered Validation and First-True Behavior

Engineering practice → ordered rules → first true branch → skipped branches → support route

Instructor key. Same layout as student copy; solutions filled in response boxes. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/42		Mastery check	secure / developing / needs support	

The sensor-kit checkout table now uses several rules in a fixed order. A kit might have more than one issue, but the route should identify the first action the team should take. Today the focus is ordered validation: trace which condition is first true and explain why later branches are skipped.

Engineering task	Route a sensor kit through an ordered checkout policy without losing evidence about skipped issues.
Computational move	Read an `if/elif/else` chain as an ordered list of checks where the first true branch wins.
Python focus	`if`, `elif`, `else`, first-true behavior, skipped branches, comparison and string-label checks.
Evidence to keep	rule order, condition result, first true branch, skipped later branch, route assigned, and rule-order risk.

Day 7 working rules

1. Python checks an `if/elif/else` chain from top to bottom.
2. The first true branch runs.
3. Later branches are skipped after the first true branch.
4. Rule order can change the route.
5. A reliable route note names the first true rule and any important skipped issue.

1 Read the ordered rules before tracing cases

recognize predict

The checkout desk now uses more than one rule. Python checks the rules in order. The first true rule assigns the route and the later rules are skipped.

```
kit_label = "S-22"
charge_percent = 76
calibration_label = "current"
borrower_initials = "MR"
damage_note = ""

if damage_note != "":
    route = "repair bench"
elif charge_percent < 80:
    route = "charge station"
elif calibration_label != "current":
    route = "calibration bench"
elif borrower_initials == "":
    route = "borrower log needed"
else:
    route = "release kit"
```

Pts.	Prompt	My evidence
1	Which rule is checked first?	**Key:** `damage_note != ""`.
1	Which rule is checked second?	**Key:** `charge_percent < 80`.
1	Which rule is true for this kit?	**Key:** `charge_percent < 80` is true because `76 < 80`; the damage rule is false.
1	Which route is assigned?	**Key:** `charge station`.

2 Trace first-true behavior

trace explain

Complete the table. Stop at the first true rule for each case.

Pts.	Damage?	Charge	Calibration	Initials	First true rule and route
2	no	76	current	MR	**Key:** First true rule: `charge_percent < 80`; route: `charge station`.
2	cracked case	76	expired	MR	**Key:** First true rule: `damage_note != ""`; route: `repair bench`. Later charge and calibration issues are skipped for routing.
2	no	91	expired	MR	**Key:** First true rule: `calibration_label != "current"`; route: `calibration bench`.
2	no	91	current	blank	**Key:** First true rule: `borrower_initials == ""`; route: `borrower log needed`.

First-true reminder

If an early rule is true, Python does not continue looking for a later, also-true rule. Your evidence should name the first true rule, not just every possible problem.

3 Explain skipped branches

explain debug

A kit may have more than one problem. Ordered routing decides which problem is handled first.

```
charge_percent = 72
calibration_label = "expired"
damage_note = ""

if damage_note != "":
    route = "repair bench"
elif charge_percent < 80:
    route = "charge station"
elif calibration_label != "current":
    route = "calibration bench"
else:
    route = "release kit"
```

Pts.	Prompt	My response
2	Which route is assigned?	**Key:** `charge station`.
2	Is the calibration rule also true for this kit? Explain.	**Key:** Yes. `calibration_label != "current"` is true because the label is `expired`.
2	Why does the calibration branch not run?	**Key:** The earlier low-charge rule is true first, so Python runs `charge station` and skips later branches.
2	What would be dangerous about reporting only the final route without the skipped issue?	**Key:** The team might charge the kit and then miss that calibration is still expired, so the kit could be treated as ready too soon.

4 Find a rule-order problem

debug test

A teammate proposes moving the release check above the damage check.

```
if charge_percent >= 80 and calibration_label == "current" and borrower_initials != "":
    route = "release kit"
elif damage_note != "":
    route = "repair bench"
else:
    route = "hold for review"
```

Pts.	Prompt	My response
2	What important condition is checked too late?	**Key:** The damage check, `damage_note != ""`.
2	Give a kit case that would be routed unsafely by the proposed order.	**Key:** A damaged but otherwise ready kit: `damage_note = "cracked case"`, `charge_percent = 90`, `calibration_label = "current"`, and initials present.
2	What route should that case receive?	**Key:** `repair bench`.
2	Write one sentence explaining the rule-order risk.	**Key:** If the release rule is checked before damage, a damaged but otherwise ready kit can be released before the repair rule is reached.

5 Constrained edit of one rule

implement debug

Only edit the calibration rule below. Do not rewrite the whole route.

```
    elif ________________________________________________:
        route = "calibration bench"
```

Pts.	Prompt	My response
3	Complete the calibration rule.	**Key:** `calibration_label != "current"`.
2	Explain why the rule should compare to `"current"`.	**Key:** The policy needs the exact acceptable label. Anything other than `current`, such as `expired`, should route to calibration.
2	Give one calibration label that should route to the calibration bench.	**Key:** `expired`.
1	Name one later branch that would be skipped if this rule is true.	**Key:** The borrower-log branch, `borrower_initials == ""`, would be skipped.

6 Transfer and support route

transfer explain

Pts.	Ordered-validation note
4	Write a note that explains how ordered validation differs from a single `if/else`. Use first true and skipped branch in your explanation. **Key:** A single `if/else` chooses between two routes from one condition. Ordered validation checks several rules top to bottom; the first true rule wins, so a low-charge and expired kit routes to `charge station` while the calibration branch is skipped for that pass.
2	Circle the support route you would use next: rule order / skipped branch / comparison / string label / written explanation. Name one next action: _____

Bridge to Day 8

Day 8 uses expected-vs-actual evidence to debug a route when the selected action does not match the rule intention.

VIP Lab Alignment

Bridge Day 7 • Contract c821cfcf7189

This page replaces the earlier wrapper-based lab bridge and follows the current top-level notebook interface.

Why this page is here

VIP Lab Day(s): 4 **Activities:** 18.13, 18.14, 18.15, 18.16, 18.17

Carry comparison and branch-order evidence into the current top-level decision cells; Activity 18.14 supplies its loop.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.13 `18-13-work` / `18-13-transfer`	**Compare With Tolerance**	Compute absolute distance between two values.
18.14 `18-14-work` / `18-14-transfer`	**Count High Readings**	Read a supplied loop without having to design it yet.
18.15 `18-15-work` / `18-15-transfer`	**Lamination Fee**	Translate three quantity intervals into ordered branches.
18.16 `18-16-work` / `18-16-transfer`	**Temperature Status**	Compare a measurement with a maximum.
18.17 `18-17-work` / `18-17-transfer`	**Pickup Window**	Calculate round-trip travel time.

Readiness check before you code

1. Identify which activity supplies a loop and state exactly what decision you are responsible for writing inside it.

   Answer and explanation: Activity 18.14 supplies the loop. Students write the strict greater-than decision and update the counter only on matching passes.

2. For one of Activities 18.13, 18.15, 18.16, or 18.17, name an equality boundary that must receive its own test.

   Answer and explanation: Accept a correct activity boundary, including the tolerance equality in 18.13, an inclusive fee boundary in 18.15, the exact +5 boundary in 18.16, or the quick-pickup time boundary in 18.17.

Notebook and submission procedure

1. Work in the provided sample and transfer cells; do not delete or recreate them.
2. Run each cell and compare fresh output with the notebook's stated result before revising.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Contract-controlled page • generated from the current VIP activity profiles and stable notebook cell IDs